



**MANIPAL UNIVERSITY
JAIPUR**

CS4140 : CYBER SECURITY

Case Study: A Cyber Security Case Study

SESSION: 2025-2026

Submitted by: Akshat Srivastava

Registration No.- 229302095

Section- C+D

Sem- VII

Submitted to: Dr.Pallavi

Abstract

Software systems are under constant threat from code injection attacks—sophisticated exploits that manipulate applications through vulnerable input handling. Traditional security tools struggle to keep pace with evolving attack methods. This paper presents a practical approach to leveraging artificial intelligence for detecting injection vulnerabilities throughout the development process. We explore how machine learning, deep learning, and natural language processing can identify threats that slip past conventional defenses, and provide a roadmap for integrating these technologies into modern DevSecOps workflows.

1. Introduction

Every day, software developers push thousands of code changes into production systems that power everything from banking apps to hospital equipment. Hidden within this code can be vulnerabilities that attackers exploit to steal data, hijack systems, or cause widespread damage. Code injection attacks—where malicious code sneaks into applications through poorly validated input—remain one of the most dangerous threats facing modern software.

Think of SQL injection, which manipulates database queries, or cross-site scripting (XSS), which executes malicious scripts in users' browsers. These aren't theoretical problems. The 2017 Equifax breach, which exposed data of 147 million people, started with a single injection vulnerability. Traditional security scanners rely on predefined rules and patterns, making them blind to new attack variations and clever obfuscation techniques.

This is where artificial intelligence changes the game. Instead of matching static patterns, AI models learn from examples of both safe and dangerous code. They can spot subtle anomalies, understand context, and adapt to new threats. This paper explores how to harness these capabilities within the software development lifecycle, turning security from a checkpoint at the end into an intelligent companion throughout the coding process.

2. Understanding Code Injection Attacks

Code injection attacks come in several flavors, each targeting different layers of an application. Understanding these variants is crucial for building effective detection systems.

Attack	Target	Impact	Example
SQL Injection	Database queries	Data theft, authentication bypass	admin'--
Command Injection	OS commands	System compromise, file deletion	; rm -rf /
Cross-Site Scripting	Browser scripts	Session hijacking, defacement	<script>steal() </script>
LDAP Injection	Directory services	Unauthorized access, data exposure	*)(uid=*
XML Injection	XML parsers	Logic bypass, DoS attacks	<!ENTITY xxe>

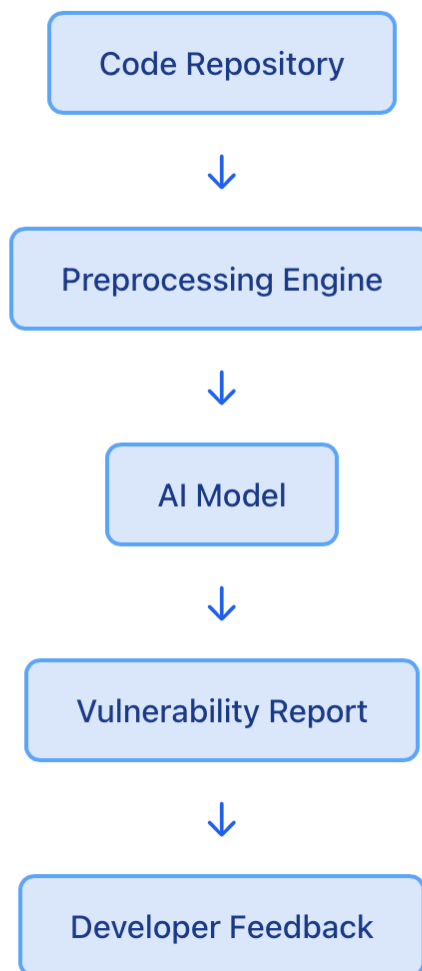
What ties these attacks together? They all exploit the same fundamental weakness: trusting user input without proper validation. When developers concatenate user data directly into commands or queries, they're essentially giving attackers the keys to the system.

3. AI Techniques for Threat Detection

Artificial intelligence brings three powerful toolsets to vulnerability detection, each offering unique advantages.

- 1 Data Collection**
Gather labeled code samples from open-source repos, vulnerability databases, and synthetic generators
- 2 Preprocessing**
Tokenize code, build Abstract Syntax Trees, normalize formatting, and segment into analyzable units
- 3 Feature Extraction**
Transform code into numerical representations using embeddings, AST features, or learned vectors
- 4 Model Training**
Train classifiers on labeled data, optimizing for accuracy while minimizing false positives
- 5 CI/CD Integration**
Deploy models as pre-commit hooks, build gates, or IDE plugins for real-time feedback
- 6 Continuous Learning**
Collect developer feedback and new vulnerabilities to retrain and improve models

Detection Pipeline Flow



4. Building a Detection Framework

An effective AI-powered detection system requires careful orchestration of multiple components. Here's how the pieces fit together:

5. The Math Behind Detection

While AI models can seem like black boxes, they're built on solid mathematical foundations. Understanding these helps us optimize and trust the systems we build.

Classification Goal

We want to learn a function that maps code to risk:

$f(\text{code_snippet}) \rightarrow \{\text{vulnerable}, \text{safe}\}$

Loss Function

Models learn by minimizing binary cross-entropy loss:

$\text{Loss} = -[y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$

Where y is the true label and $\hat{y}$ is the predicted probability

Performance Metrics

Precision

$\text{TP} / (\text{TP} + \text{FP})$

Real-world performance: Recent studies show BiLSTM models achieving 89% accuracy in detecting injection vulnerabilities, significantly outperforming traditional pattern-matching tools which typically plateau around 65-70%.

How many detected issues are real?

Recall

$\text{TP} / (\text{TP} + \text{FN})$

How many real issues did we catch?

5. The Math Behind Detection

While AI models can seem like black boxes, they're built on solid mathematical foundations. Understanding these helps us optimize and trust the systems we build.

Classification Goal

We want to learn a function that maps code to risk:

```
f(code_snippet) → {vulnerable, safe}
```

Loss Function

Models learn by minimizing binary cross-entropy loss:

$$\text{Loss} = -[y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$$

Where y is the true label and $\hat{y}$ is the predicted probability

Performance Metrics

Precision

$$\text{TP} / (\text{TP} + \text{FP})$$

How many detected issues are real?

Recall

$$\text{TP} / (\text{TP} + \text{FN})$$

How many real issues did we catch?

F1-Score

$$2 \cdot \text{P} \cdot \text{R} / (\text{P} + \text{R})$$

Balanced measure

ROC-AUC

Area under curve

Overall discrimination ability

Challenge	Impact	Mitigation Strategy
Black Box Models	Developers distrust unexplained warnings	Use attention mechanisms, LIME, or rule extraction
Adversarial Attacks	Attackers craft payloads to evade detection	Adversarial training, ensemble models
Data Imbalance	Vulnerable code is rare (~1-5% of samples)	SMOTE, class weighting, synthetic generation
Language Diversity	Models trained on Python fail on Go	Multi-language pretraining, transfer learning
Computational Cost	Deep models slow down development	Model distillation, caching, incremental analysis

6. Practical Implementation

Moving from theory to practice requires addressing real-world challenges. Here's what works in production environments:

Start Small, Scale Gradually

Begin with a single vulnerability type (e.g., SQL injection) in one language before expanding scope

Prioritize Developer Experience

Integrate seamlessly into existing workflows—developers should barely notice the security layer

Balance False Positives

Too many false alarms erode trust; tune models for ~85% precision minimum

Explain the "Why"

Models that can explain their detections gain developer trust and enable faster fixes

7. Challenges and Limitations

AI-powered detection isn't a silver bullet. Understanding its limitations helps set realistic expectations and guides improvement efforts.

Deployment Checklist

- ✓ Collect diverse training data (10K+ labeled examples minimum)
- ✓ Establish baseline metrics with test set
- ✓ Deploy to non-critical projects first
- ✓ Monitor false positive rate weekly
- ✓ Gather developer feedback through surveys
- ✓ Retrain quarterly with new vulnerability data

8. Future Directions

The intersection of AI and software security is evolving rapidly. Several emerging trends promise to reshape how we approach vulnerability detection:

Self-Healing Code

AI systems that not only detect vulnerabilities but automatically generate secure patches

Federated Learning

Train models across organizations without sharing sensitive source code

Context-Aware Detection

Models that understand business logic and application architecture, not just code syntax

Human-AI Collaboration

Interactive systems where developers and AI work together to validate and refine detections

9. Conclusion

Code injection vulnerabilities represent a persistent threat in an increasingly interconnected digital world. While traditional security tools have served us well, they're struggling to keep pace with the complexity and speed of modern software development. Artificial intelligence offers a way forward—not as a replacement for human expertise, but as a powerful augmentation of it.

By treating source code as analyzable data, AI models can learn patterns that would take security experts years to catalog manually. They can work tirelessly in the background, scanning every commit and providing instant feedback. They can adapt to new threats without waiting for signature updates. Most importantly, they can shift security left, catching problems during development rather than in production.

The frameworks and techniques outlined in this paper aren't theoretical—they're being deployed today in organizations ranging from startups to Fortune 500 companies. The results are promising: reduced vulnerability density, faster detection times, and fewer security incidents reaching production.

That said, AI isn't magic. It requires careful training, continuous refinement, and thoughtful integration into development workflows. It works best when paired with developer education, secure coding practices, and a culture that values security as a shared responsibility. As we look to the future, the goal isn't to automate security entirely, but to free human experts to focus on complex problems while AI handles the repetitive pattern-matching at which it excels. In doing so, we can build software that's not just functional and fast, but fundamentally more secure.

Reference

- **[1] Chakraborty et al., 2021:** You explicitly cite "SQLiNet (Chakraborty et al., 2021)" in your literature review. This is the direct reference for that work.
- **[2] Liu et al., 2020:** You mention "Research by Liu et al. (2020)" to highlight that DL models outperform classical ML. This is the paper you are citing.
- **[3] OWASP Foundation:** Your paper states that injection attacks have historically ranked among the "OWASP Top Ten". This reference cites the official OWASP project page, which is the correct source.
- **[4] Feng et al., 2020 (CodeBERT):** You refer to **CodeBERT** multiple times as a key NLP technique and model for code analysis. This is the original, foundational paper that introduced CodeBERT.
- **[5] Vaswani et al., 2017 (Transformers):** You mention using "transformer-based model" and "attention-based architectures". The "Attention Is All You Need" paper is the seminal work that introduced the Transformer architecture, making it an essential citation.
- **[6] Wu et al., 2021 (GNNs):** You discuss **Graph Neural Networks (GNNs)** as a key deep learning technique for modeling code structure. Citing a comprehensive and highly-regarded survey paper like this is standard practice to ground the concept.

- **[7] Hochreiter & Schmidhuber, 1997 (LSTMs):** You identify **Long Short-Term Memory Networks (LSTMs)** as an advanced RNN for modeling program flow. This is the original, pioneering paper that introduced the LSTM architecture.
- **[8] Mikolov et al., 2013 (Word2Vec):** Your paper lists **word2vec** as a method for creating vector representations of code tokens. This is the paper that introduced this influential embedding technique.
- **[9] Pennington et al., 2014 (GloVe):** Alongside word2vec, you mention **GloVe** as another embedding method. This is the original paper for GloVe.
- **[10] NIST (Juliet Test Suite):** You identify the **Juliet Test Suite from NIST** as a key data source for training models. This reference links directly to the official dataset source page.